

- [ExPrep Rich Content & Question Schema Documentation](#)
 - [1. Rich Content Block \(RichContentDto\)](#)
 - [JSON Structure](#)
 - [Supported Media Types](#)
 - [2. Question Types & Data Structures](#)
 - [A. Choice Questions \(SINGLE_CHOICE, MULTIPLE_CHOICE\)](#)
 - [B. Fill in the Blank \(FILL_IN_BLANK\)](#)
 - [C. True / False \(TRUE_FALSE\)](#)
 - [D. Essay \(ESSAY\)](#)
 - [E. Essay with Sub-questions \(ESSAY_WITH_SUB\)](#)
 - [F. Short Answer \(SHORT_ANSWER\)](#)
 - [G. Matching \(MATCHING\)](#)
 - [H. Ordering / Sequence \(ORDERING\)](#)
 - [I. Calculation \(CALCULATION\)](#)
 - [J. Diagram Labeling \(DIAGRAM_LABELING\)](#)
 - [K. Theory with Objectives \(THEORY_WITH_OBJECTIVES\)](#)
 - [3. Explanations \(ExplanationDto\)](#)
 - [4. Multi-part / Media Uploads](#)

ExPrep Rich Content & Question Schema Documentation

This document provides a comprehensive guide for frontend developers on how to interact with the ExPrep question system. It covers the **Rich Content Block** system and the structures for **every supported question type**.

1. Rich Content Block (**RichContentDto**)

Rich content blocks are the foundation of questions, options, and explanations. They allow a mix of text, media, and structured data.

JSON Structure

A field like **questionText** or **solution** is always an **array of objects**.

```
[
  {
    "type": "text",
    "value": "What is the capital of France?",
    "style": { "bold": true, "fontSize": "large" }
  },
  {
    "type": "image",
    "url": "https://res.cloudinary.com/...",
    "alt": "Eiffel Tower"
  }
]
```

Supported Media Types

Type	Main Content Field	Supported Extensions
text	value	Plain text
markdown	content	GFM (GitHub Flavored Markdown)
latex	value	Mathematical equations (KaTeX syntax)
image	url	jpg, png, webp, svg
audio	url	mp3, wav
video	url	mp4, youtube links
table	headers, rows	2D JSON array for headers and rows
diagram	imageUrl	Interactive diagrams with annotations
list	items	Array of strings

2. Question Types & Data Structures

Every question must have a `questionType`. Depending on this type, a corresponding data field must be populated.

A. Choice Questions (`SINGLE_CHOICE`, `MULTIPLE_CHOICE`)

Used for standard multiple-choice questions.

- **Field:** `options` (Array)
- **Correct:** `correctAnswer` (string, e.g., "A") or `correctAnswers` (array, e.g., ["A", "C"])

Example Options:

```
"options": [  
  {  
    "id": "A",  
    "content": [{ "type": "text", "value": "Paris" }],  
    "isCorrect": true  
  },  
  {  
    "id": "B",  
    "content": [{ "type": "text", "value": "London" }],  
    "isCorrect": false  
  }  
]
```

B. Fill in the Blank (**FILL_IN_BLANK**)

Used for questions where students type answers into placeholders.

- **Field:** `fillInBlankData`
- **Mechanism:** Use `[[1]]`, `[[2]]` in the `template` string to represent blanks.

Example:

```
"fillInBlankData": {  
  "template": "The capital of [[1]] is [[2]].",  
  "blanks": [  
    {  
      "id": "1",  
      "acceptableAnswers": ["France", "French Republic"],  
      "inputType": "text"  
    },  
    {  
      "id": "2",  
      "acceptableAnswers": ["Paris"],  
      "hint": "City of Light"  
    }  
  ]  
}
```

C. True / False (TRUE_FALSE)

Simple boolean question.

- **Field:** trueFalseData

Example:

```
"trueFalseData": {  
  "correctAnswer": true,  
  "justificationRequired": false  
}
```

D. Essay (ESSAY)

Long-form text response.

- **Field:** essayData

Example:

```
"essayData": {  
  "minWords": 100,  
  "maxWords": 500,  
  "expectedPoints": ["History", "Geography", "Culture"],  
  "rubric": [  
    { "criterion": "Grammar", "maxMarks": 5 },  
    { "criterion": "Content", "maxMarks": 10 }  
  ]  
}
```

E. Essay with Sub-questions (ESSAY_WITH_SUB)

Composite question with multiple sub-parts.

- **Field:** essayWithSubData

Example:

```

"essayWithSubData": {
  "mainQuestion": [{ "type": "text", "value": "Analyze Nigeria's economy." }],
  "subQuestions": [
    {
      "subId": "a",
      "questionText": [{ "type": "text", "value": "Name 3 exports." }],
      "marks": 5
    },
    {
      "subId": "b",
      "questionText": [{ "type": "text", "value": "Discuss inflation." }],
      "marks": 10
    }
  ]
}

```

F. Short Answer (**SHORT_ANSWER**)

Quick text/number response.

- **Field:** **shortAnswerData**

Example:

```

"shortAnswerData": {
  "acceptableAnswers": ["Newton", "Isaac Newton"],
  "caseSensitive": false,
  "answerFormat": "text"
}

```

G. Matching (**MATCHING**)

Connecting items from two columns.

- **Field:** **matchingData**

Example:

```

"matchingData": {
  "leftColumn": [
    { "id": "L1", "text": "Oxygen" },
    { "id": "L2", "text": "Gold" }
  ],

```

```
"rightColumn": [
  { "id": "R1", "text": "O" },
  { "id": "R2", "text": "Au" }
],
"correctMatches": ["L1-R1", "L2-R2"]
}
```

H. Ordering / Sequence (**ORDERING**)

Arranging items in a specific sequence.

- **Field:** **orderingData**

Example:

```
"orderingData": {
  "items": [
    { "id": "1", "text": "Wake up" },
    { "id": "2", "text": "Brush teeth" },
    { "id": "3", "text": "Have breakfast" }
  ],
  "correctOrder": ["1", "2", "3"]
}
```

I. Calculation (**CALCULATION**)

Mathematical problems with step-by-step solutions.

- **Field:** **calculationData**

Example:

```
"calculationData": {
  "problem": "Solve  $2x + 5 = 15$ ",
  "steps": [
    { "step": "Subtract 5", "formula": " $2x = 10$ " },
    { "step": "Divide by 2", "formula": " $x = 5$ " }
  ],
  "finalAnswer": 5,
  "precision": 0
}
```

J. Diagram Labeling (DIAGRAM_LABELING)

Identifying parts of an image.

- **Field:** `diagramLabelingData`

Example:

```
"diagramLabelingData": {
  "diagramUrl": "https://...",
  "labels": [
    { "id": "1", "label": "Nucleus", "x": 50, "y": 50 },
    { "id": "2", "label": "Mitochondria", "x": 80, "y": 30 }
  ]
}
```

K. Theory with Objectives (THEORY_WITH_OBJECTIVES)

Hybrid question consisting of a theory part and an objectives (MCQ) part.

- **Field:** `theoryWithObjectivesData`

Example:

```
"theoryWithObjectivesData": {
  "theoryPart": {
    "content": [{ "type": "text", "value": "Define Photosynthesis." }],
    "responseType": "SHORT_ANSWER",
    "marks": 5
  },
  "objectivesPart": {
    "content": [{ "type": "text", "value": "Which gas is released?" }],
    "options": [
      { "id": "A", "content": [{"type": "text", "value": "Oxygen"}] }
    ],
    "correctAnswer": "A",
    "marks": 1
  }
}
```

3. Explanations (ExplanationDto)

The `explanation` field is a structured object, not just a string.

Field	Type	Usage
<code>solution</code>	<code>RichContentDto[]</code>	The detailed answer.
<code>workingSteps</code>	<code>Array</code>	Sequential steps for math/science.
<code>keyPoints</code>	<code>string[]</code>	Bullet points for quick recall.
<code>commonMistakes</code>	<code>string[]</code>	Pitfalls to avoid.
<code>tips</code>	<code>string[]</code>	Exam strategies.

4. Multi-part / Media Uploads

[!IMPORTANT] When uploading images/videos while creating a question, you MUST send the metadata (the entire JSON) as a string.

Payload structure for Multipart:

- `questionText`: "[{"type":"text","value":"..."}]" (Stringified JSON)
- `explanation`: "{"solution":[...]}" (Stringified JSON)
- `questionImages`: (Binary File)
- `optionImages`: (Binary File)

The backend will automatically:

1. Parse the stringified JSON.
2. Upload the binary files to Cloudinary.
3. Replace placeholder objects in your JSON with the actual Cloudinary URLs and Public IDs.